Java Programming

CPT111 – Lab 8

Brennan Jones

Xi'an Jiaotong-Liverpool University
西交利物浦大学

CPT111 Java Programming

Lab 8

More Objects and Inheritance

Clock + Getters

- In the exercises Week 8
  - we will subclass the class Clock from Week 6
  - but with **<u>you must add these getters into it</u>**

```
// Getters
public int getHours() {
    return hours;
}
public int getMinutes() {
    return minutes;
}
```

  - complete Clock.java with your Week 6 code ***first***!

Alarm Clock

- AlarmClock is a subclass of Clock
  - you can still use the inherited methods such as toString()

- In addition, you can now also set up an alarm to alert you in a specified time
  - using the default or a chosen message
  - by overriding Clock's method tick()

Exercise #8.1 AlarmClock Constructor 1

- Complete the first constructor of the class AlarmClock
- It takes four arguments: h, m, alarmHours, alarmMinutes
  - creates a new AlarmClock object whose initial time is h hours and m minutes
  - sounds an alarm at alarmHours hours and alarmMinutes minutes, with the default sound "Beep beep beep beep!"

- Test cases:

```
AlarmClock ac1 = new AlarmClock(5, 58, 6, 0);
ac1.tick();
ac1.tick();                    → Beep beep beep beep!
System.out.println(ac1);       → 06:00
```

Exercise #8.2 AlarmClock Constructor 2

- Complete the second constructor of the class AlarmClock
- Overloading the first constructor, it now takes five arguments: h, m, alarmHours, alarmMinutes, and alarmSound
  - creates a new AlarmClock object whose initial time is h hours and m minutes
  - sounds an alarm at alarmHours hours and alarmMinutes minutes, and sets the sound to alarmSound
- Test cases:

```
AlarmClock ac2 = new AlarmClock(14, 29, 14, 30, "Wake Up! The Hero! Kamen Rider!");
ac2.tick();                                  →  Wake Up! The Hero! Kamen Rider!
```

Exercise #8.3 AlarmClock Tick

- Complete the method tick of the class AlarmClock
- It overrides the method tick of Clock and adds 1 minute to the time on this alarm clock
  - In addition, it sounds (prints) the alarm at the specified time.

- Test cases:

```
AlarmClock ac1 = new AlarmClock(5, 58, 6, 0);
ac1.tick();
ac1.tick();                                  → Beep beep beep beep!
System.out.println(ac1);                     → 06:00
AlarmClock ac2 = new AlarmClock(14, 29, 14, 30, "Wake Up! The Hero! Kamen Rider!");
ac2.tick();                                  →  Wake Up! The Hero! Kamen Rider!
```

Cuckoo Clock

- CuckooClock is a subclass of Clock
  - you can still use the inherited methods such as toString()

- In addition, the CuckooClock can print "Cuckoo!" at the start of every hour
  - as many as the current hours, no matter it is morning or night

Exercise #8.4 CuckooClock Constructor

- Complete the constructor of the class CuckooClock
- It takes two arguments: h and m
  - and creates a new CuckooClock object whose initial time is h hours and m minutes

- Test cases:

```
CuckooClock cc1 = new CuckooClock(0, 58);
cc1.tick();
cc1.tick();                                  → Cuckoo!
System.out.println(cc1);                     → 01:00
CuckooClock cc2 = new CuckooClock(13, 59);
cc2.tick();                                  → Cuckoo!↲Cuckoo!
```

Exercise #8.5 CuckooClock Tick

- Complete the method tick of the class CuckooClock
- It overrides the method tick of Clock and adds 1 minute to the time on this Cuckoo clock
- In addition, it prints "Cuckoo!" at the start of every hour
  - It prints one time for each hour
  - Whether it is morning or night does **not** change the number of times it prints

- Test cases:

```
CuckooClock cc1 = new CuckooClock(0, 58);
cc1.tick();
cc1.tick();                                  → Cuckoo!
System.out.println(cc1);                     → 01:00
CuckooClock cc2 = new CuckooClock(13, 59);
cc2.tick();                                  → Cuckoo!↲Cuckoo!
```

Halloween Clock

- HalloweenClock is a subclass of Clock
  - you can still use the inherited methods such as toString()

- In addition, whenever ***any*** current instances of HalloweenClock have ticked three times
  - the latest ticking object will also print "Halloween!"

Exercise #8.6 HalloweenClock Constructor

- Complete the constructor of the class HalloweenClock
- It takes two arguments: h and m
  - and creates a new HalloweenClock object whose initial time is h hours and m minutes
- Test cases:

```
HalloweenClock hc1 = new HalloweenClock(1, 0);
HalloweenClock hc2 = new HalloweenClock(2, 0);
hc1.tick();
hc2.tick();
hc2.tick();                    → Halloween!
HalloweenClock hc3 = new HalloweenClock(3, 30);
hc1.tick();
hc2.tick();
hc3.tick();                    → Halloween!
System.out.println(hc3);       → 03:31
```

Exercise #8.7 HalloweenClock Tick

- Complete the method tick of the class HalloweenClock
- It overrides the method tick of Clock, adds 1 minute to the time on this Halloween clock
  - and if ***any*** Halloween clocks have ticked three times, prints "Halloween!"
- Test cases:

```
HalloweenClock hc1 = new HalloweenClock(1, 0);
HalloweenClock hc2 = new HalloweenClock(2, 0);
hc1.tick();
hc2.tick();
hc2.tick();                    → Halloween!
HalloweenClock hc3 = new HalloweenClock(3, 30);
hc1.tick();
hc2.tick();
hc3.tick();                    → Halloween!
System.out.println(hc3);       → 03:31
```

Lab 8 Challenge – Extending the `Counter` Class

- You are provided an existing class, `Counter` (in `Counter.java`), that represents a simple counter.
- Counter has the following instance variables:
  - `name`: the name of the counter (`String`)
  - `value`: a value that is always `>= 0` (`int`)
- As well as the following behaviours (instance methods):
  - `increment()`: add `1` to the counter's `value`
  - `reset()`: resets the counter's `value` to `0`
  - `toString()`: returns a `string` representation of the counter
- Your task is to **create a new subclass**, `BoundedCounter`, that **prevents the value from exceeding a maximum limit**.

Lab 8 Challenge – Requirements

- Class declaration: `public class BoundedCounter extends Counter`
- New instance variable: `private int max;`
- A **no-argument constructor** that calls `super()` and sets `max = 10`.
- A **full constructor**, `public BoundedCounter(String name, int initialValue, int max)`, that:
  - Calls `super(name, initialValue)`.
  - If `max < 0`, set `this.max` to `0`.
  - If the starting `value` is greater than `max`, store the smaller of the two using logic inside your constructor (since `value` is `private` in `Counter`, you cannot access it directly—use available `public` methods).
- New methods:
  - `public int getMax()`: A getter for `max`.
  - `public boolean isAtMax()`: Returns `true` if the counter's `value` equals `max`.
- Override `increment()` so that it stops increasing once the counter's `value` reaches `max`.
- Override `toString()` to return a `String` formatted like this:

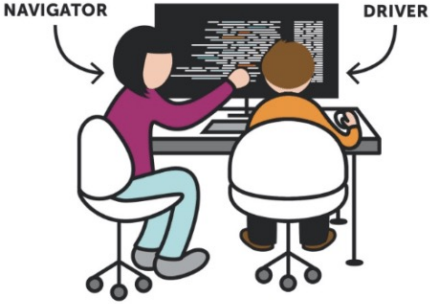
  `BoundedCounter(name="Study reps", value=5, max=10)`

Lab 8 Challenge – Example Test Cases

```
BoundedCounter a = new BoundedCounter();
System.out.println(a);              // BoundedCounter(name="untitled", value=0, max=10)
a.increment();
System.out.println(a.getValue());   // 1
System.out.println(a.isAtMax());    // false

BoundedCounter b = new BoundedCounter("Study reps", 12, 10);
System.out.println(b);              // BoundedCounter(name="Study reps", value=10, max=10)
b.increment();                      // stays 10
System.out.println(b.getValue());   // 10
System.out.println(b.isAtMax());    // true
```

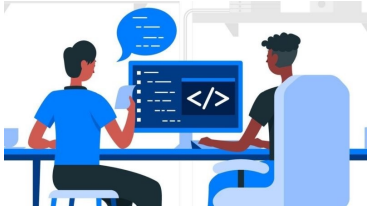
You are encouraged to try Pair Programming while completing this week's challenge!



- What is **Pair Programming**?
  - **Driver** designs and writes the codes;
  - **Observer/Navigator** reviews and suggests improvements

- Pair programming not only **improves the code quality**
  - but also **enhances learning** – learn faster by discussing, debating, and receiving explanations for concepts in real time, helps in clarifying doubts and understanding complex programming concepts more effectively
  - **knowledge sharing** – learn from their pair's experience; by working together exposes them to different problem-solving approaches, coding techniques, and best practices
  - **problem-solving, teamwork and communication skills**

Effective Pair Programming



- **Effective Communication**: encourage an open dialogue where both participants feel comfortable expressing their ideas and asking questions, explain your thought process as you code and discuss potential solutions
- **Switch Roles Regularly**: at least once as the driver and the observer during the lab
- **Stay Focused**: avoid distractions during the session, focus on the task at hand
- **Utilize Breaks**: take short breaks when needed to refresh your mind
- **Set Goals**: outline what you aim to achieve in each session, clearly define tasks and objectives, so that you maintain a sense of direction and purpose
- **Be Patient and Respectful**: beginning programmers may have different paces or styles of learning, so be patient and respectful towards each other's ideas and approaches – encourage a **positive and supportive atmosphere**
- **Feedback and Learning**: provide constructive feedback after the session, discuss what went well and what could be improved, encourage learning from each other’s strengths and weaknesses

Thank you for your hard work!

- If you finish early, feel free to use the remaining time to work on other online exercises.
  - Use this opportunity to practice your new skills in testing your code!
- You may leave the lab once you have completed the Lab 8 Challenge and it has been ASSESSED by a TA!
  - Please note that the **Lab 8 Challenge** is **graded** and contributes to your CW2 marks.